

Informe sobre los principios SOLID

Jorman Noriega

25 de agosto de 2026

Resumen

Este informe explica los cinco principios SOLID mediante ejemplos en Java. Cada ejemplo presenta el problema original y una solución basada en una abstracción sencilla.

1. Introducción

SOLID reúne cinco principios para diseñar programas orientados a objetos más fáciles de mantener, extender y probar. No son reglas para crear más clases sin motivo: ayudan a separar responsabilidades y a controlar las dependencias.

El proyecto se compila directamente con Java 11. Los importes del ejercicio de pedidos se representan con `BigDecimal`, apropiado para valores monetarios.

2. S: Responsabilidad única

Una clase debe tener una sola responsabilidad y una sola razón para cambiar. En el ejemplo incorrecto, `PedidoIncorrecto` mezcla datos, cálculo y persistencia. La solución separa esas tareas: `Pedido` mantiene sus productos, `CalculadoraPedido` calcula el total y `PedidoRepository` se ocupa de guardar.

```
Pedido pedido = new Pedido();
pedido.agregarProducto(new Producto("Cuaderno", new BigDecimal("10000")));
BigDecimal total = new CalculadoraPedido().calcularTotal(pedido);
new PedidoRepository().guardar(pedido);
```

Si cambia la forma de guardar, no es necesario modificar el modelo del pedido.

3. O: Abierto/cerrado

Una clase debe estar abierta para extenderse, pero cerrada para modificarse. El ejemplo incorrecto usa una cadena y condicionales para decidir el tipo de pago. Cada nuevo método obliga a modificar la función.

La solución define `MetodoPago` y hace que `ProcesadorPago` dependa de esa interfaz:

```

public class ProcesadorPago {
    private final MetodoPago metodoPago;

    public ProcesadorPago(MetodoPago metodoPago) {
        this.metodoPago = metodoPago;
    }

    public void procesar(double valor) {
        metodoPago.pagar(valor);
    }
}

```

Se pueden añadir TarjetaPago, PaypalPago, NequiPago u otra implementación sin modificar el procesador.

4. L: Sustitución de Liskov

Una subclase debe poder utilizarse donde se espera su clase padre sin romper el contrato. En el ejemplo incorrecto, el empleado de medio tiempo lanza una excepción al calcular su salario.

En la solución, todos los tipos implementan la operación heredada de forma válida:

```

abstract class Empleado {
    public abstract double calcularSalario();
}

class EmpleadoMedioTiempo extends Empleado {
    public double calcularSalario() { return 1000000; }
}

```

También se incluye EmpleadoPorComision. Una lista de Empleado puede contener los tres tipos y calcular sus salarios sin conocer sus detalles.

5. I: Segregación de interfaces

Una clase no debe estar obligada a implementar métodos que no necesita. La interfaz incorrecta TrabajadorIncorrecto obliga al robot a comer y descansar.

La solución divide el contrato en Trabajable, Comible y Descansable. El empleado implementa las tres, mientras que el robot solo implementa Trabajable:

```

class Robot implements Trabajable {
    public void trabajar() {
        System.out.println("El robot esta trabajando");
    }
}

```

El programa utiliza métodos que reciben únicamente la capacidad requerida, como el método asignarTrabajo. Esto evita que el robot no dependa de las otras interfaces.

6. D: Inversión de dependencias

Las clases de alto nivel deben depender de abstracciones, no de detalles concretos. En el ejemplo incorrecto, `PedidoService` crea directamente un `EmailNotificador`.

La solución recibe un `Notificador` por el constructor:

```
public PedidoService(Notificador notificador) {  
    this.notificador = Objects.requireNonNull(  
        notificador, "El notificador es obligatorio");  
}
```

El mismo servicio funciona con correo, WhatsApp u otro canal sin modificar el servicio.

7. Conclusión

Los cinco ejemplos muestran que SOLID mejora la mantenibilidad cuando se aplica a problemas concretos: separar el pedido de su cálculo, extender pagos mediante interfaces, respetar contratos de herencia, dividir interfaces grandes e inyectar dependencias.